TeSpec: Testing Stateful C Specifications through Symbolic Verification

Anonymous Authors

***Abstract*—Specification testing for C commonly compiles logical assertions into executable checks and evaluates them during concrete program runs. Proof-oriented functional specifications can contain quantified properties and abstract relations that still require deduction after the input data are fixed. We present TESPEC, which formulates specification testing as instance-specialized symbolic verification. It reuses a verifier's symbolic execution, logical heap model, and proof infrastructure to check the original assertions without a native test execution or a separate executable monitor for residual properties. Partial bindings define the input domain. Symbolic execution follows program bodies and loops; annotations generate obligations on state copies. Joint heap/value normalization and exact local frames preserve the information needed by later checks. We mechanize observation noninterference and nonvacuous acceptance under explicit coverage interfaces. A least-root instance checks all 27 obligations, including a quantified intermediate assertion. The original quantified postcondition is rejected by the tested CN runtime-testing configurations, while a proved finite rewrite passes Fulminate. Complete proof-group checks take 124.45–140.12 seconds, excluding construction. The method unifies instance testing and verification around the same symbolic execution and proof logic, making mathematical functional specifications directly usable as test criteria.**

***Index Terms*—Specification testing, symbolic execution, separation logic, proof checking**

I. INTRODUCTION

A functional specification relates what a C program computes to mathematical values, quantified properties, and abstract data models. Verification can establish such a relation by proof. Testing usually needs a procedure that decides it on each observed state. For proof-oriented specifications, constructing that procedure can require reformulating the property and proving the reformulation correct. We formulate specification testing as instance-specialized symbolic verification: the verifier executes the program symbolically on a partially bound input domain and checks the original assertion, including instances whose truth requires deduction. This reuses both the execution engine and the functional proof oracle.

For heap-manipulating C, the criterion must account for ownership, aliasing, shared writes, and the states inside loops and calls. A final return value can be correct even when an intermediate assertion is false. The test therefore needs both program-derived states and a proof-capable oracle for those states.

Existing work provides two foundations. Fulminate translates CN separation-logic contracts into runtime ownership and functional checks [1]. Bennet supplies generated heap inputs [2]; the subsequent Code-Specify-Test-Debug-Prove workflow adds loop-invariant and lemma testing and explicit ghost arguments [3]. This testing path compiles supported assertions into executable checks and runs the instrumented program on concrete heaps. COINS takes a proof-based route: it specializes declarative Rocq specifications on trusted examples and checks the resulting obligations [4]. The opportunity is to perform both state generation and property checking inside the verification infrastructure. Symbolic execution supplies the heap states, computation reduces the instantiated assertions, and proofs establish residual functional relations. This removes the need for a native test harness and an executable monitor for the residual property.

Table I locates this design relative to the closest approaches.

a) The obstacle: specializing data does not decide the assertion.: A verification assertion can contain an unbounded quantifier or an abstract relation whose instance still requires deduction. Consider a routine that replaces an owned integer n by its least nonnegative rounded-up square root r. One specification is

$$0 \le r \ \wedge \ \forall k \in \mathbb{Z}.\, 0 \le k \Rightarrow (r \le k \Leftrightarrow n \le k^2).$$

Binding $n = 13$ determines the loop computation and its result $r = 4$, while leaving a quantified mathematical proposition to check. In our pinned CN testing configurations, the original postcondition is rejected because an upper quantifier bound cannot be inferred. Fulminate accepts a proved equivalent finite reformulation. TESPEC checks the original assertion through Rocq, allowing the verification formula itself to serve as the instance oracle.

b) The challenge: observing a contract must preserve execution.: Symbolic verification commonly uses invariants and callee contracts to summarize execution. In a specification test, these summaries are themselves candidates under evaluation. Suppose a helper increments a cell from 3 to 4 but promises only an existential final value. Executing the helper retains the value 4; replacing its effects with that weak summary loses the information needed by the next check. Likewise, assuming a candidate invariant can exclude the very state that violates it. A proof-capable test oracle therefore needs a precise separation between program transitions and annotation observations.

c) Testing through symbolic verification.: TESPEC specializes a precondition with typed partial bindings. The result is a domain of initial heaps whose data can be fixed while addresses remain symbolic. It first checks that this domain is inhabited, then executes actual bodies, branches, and loops. At each selected assertion, invariant visit, or call boundary, it generates an obligation on a copy of the current state and continues

TABLE I
HOW THE CLOSEST APPROACHES OBTAIN STATES AND ESTABLISH PROPERTIES. THE COMPARISON DISTINGUISHES EXECUTION AND ORACLE CONSTRUCTION; CN ALREADY SHARES SPECIFICATION SYNTAX ACROSS TESTING AND VERIFICATION.

Approach	State or example source	Property-checking route	Scope of a successful result
CN runtime testing [1], [3]	Concrete runs of instrumented C	Executable checks compiled from supported assertions	Observed concrete executions
COINS [4]	Trusted input–output examples	Proof of the instantiated specification	The supplied examples
QCP verification [5]	Symbolic execution with modular summaries	Automatic reasoning and Rocq proofs	Full specified precondition domain
TESPEC	Symbolic execution of bodies on partially bound heap domains	Original logical assertions; computation and checked proofs	Covered instance domain and selected internal observations

from the original state. Computation and automatic reasoning simplify the obligation; residual mathematical properties remain in the verification logic for checked proof completion. The proof controller binds certificates to the complete original goal interface and its admitted assumptions.

This design yields a test over a symbolic domain: successful checking covers all feasible executions represented by the completed execution tree. The technical work is to preserve that meaning across state normalization, local-storage framing, call entry and return, and proof completion. QCP supplies symbolic execution, separation-logic reasoning, and the Rocq backend [5]. TESPEC organizes these capabilities around execution-derived observations of candidate specifications.

We make three contributions:

1) **Specification testing as specialized symbolic verification.** We reuse symbolic execution to generate test states and the verification logic to check functional assertions. Tests share the verifier's heap model, logical definitions, and proof libraries. Partial logical bindings define heap domains with an explicit existence check; a boundary schedule selects observations along actual executions. We mechanize observation erasure and lifting, and nonvacuous acceptance under explicit execution-coverage and obligation-generation interfaces (Section III).
2) **A checking discipline for stateful proof oracles.** We connect copied-state observations, exact local frames, and joint heap/value normalization to a common VC interface. Complete-goal replay and assumption auditing support proof completion in the original logic (Section IV).
3) **An evidence-based account of capability and cost.** Matched contract configurations, complete proof groups, and mechanism probes show where retaining the original assertion helps and what it costs. The least-root observation instance checks all 27 obligations. Eleven paired contract configurations and seven complete proof groups characterize automatic feedback, proof completion, and replay cost (Section V).

The resulting workflow lets developers test logical instances before completing a universal proof, while retaining the functional specification and its proof library. Testing and verification differ in the domain they establish; both use the same logical property. This is especially useful for quantified and relational contracts whose instances can be proved with existing lemmas.

II. A RUNNING EXAMPLE

Consider a routine that replaces an owned integer by its least nonnegative integer square root rounded upward. The contract relates the initial value x, the returned value, and the final cell contents. Its imported Rocq proposition is

$$\mathsf{LeastRoot}(n, r) \equiv 0 \le r \wedge \forall k \in \mathbb{Z}.\ 0 \le k \Rightarrow (r \le k \Leftrightarrow n \le k^2). \tag{1}$$

Listing 1 shows the C body and abbreviated annotations of the archived instance. We use $p \mapsto_{\mathrm{int}} v$ for typed cell ownership; the artifact uses QCP's `data_at` notation.

```
int root_cell(int *p)
/*@ With x
    Require 0 <= x && x <= 100 &&
            data_at(p, int, x)
    Ensure least_root(x, __return) &&
           data_at(p, int, __return) */
{
  int n = *p;
  int r = 0;
  /*@ Inv n == x && 0 <= r && r <= 10 &&
      (r == 0 || (r-1)*(r-1) < n) &&
      data_at(p, int, x) */
  while (r*r < n) { r = r + 1; }
  /*@ Assert least_root(n, r) &&
              data_at(p, int, x) */
  *p = r;
  return r;
}
```

Listing 1. Least-root instance with an intermediate observation. The imported relation is Equation (1).

a) A logical instance.: The binding $\theta = \{x \mapsto 13\}$ fixes the logical input and retains the address p. The spatial precondition describes the cell at that symbolic address. Loop execution produces $r = 0, 1, 2, 3, 4$ at its five invariant boundaries; then the body writes 4 and returns 4. The final observation is

$$\mathsf{LeastRoot}(13, 4) \wedge p \mapsto_{\mathrm{int}} 4.$$

Local scalar storage is handled by the observation frame described in Section IV-B. The imported relation remains

a proposition in the verification logic. Automatic execution establishes the entry and safety phases, and proof completion checks the remaining functional and observation obligations against their original generated statements.

b) Testing an intermediate specification.: Replacing the assertion by $\mathsf{LeastRoot}(n, r-1)$ changes the observation while leaving the C body and final contract intact. At that boundary the program still has $n = 13, r = 4$, so the candidate asks for $\mathsf{LeastRoot}(13, 3)$. Instantiating its universal quantifier with $k = 3$ exposes the contradiction $13 \leq 9$. The paired artifact proves the negation of the complete generated assertion VC and checks the other 26 obligations. This archived negative proof records the falsity of the quantified goal. Its execution verdict remains unresolved until reachability evidence is admitted, as detailed in Section III-D. Both variants follow the same program computation.

c) Original assertions and executable reformulations.: The pinned Fulminate, Bennet, and Darcy configurations reject the original quantified *postcondition* because they cannot infer an upper bound for k. An executable reformulation is

$$0 \leq r \wedge n \leq r^2 \wedge \big(r = 0 \vee (r-1)^2 < n\big). \tag{2}$$

The artifact proves the equivalence of Equations (1) and (2) over mathematical integers; the C contract bounds inputs to control machine arithmetic. The reformulated postcondition passes Fulminate on the same fixture. This comparison identifies the concrete benefit: TESPEC can check the original verification assertion, while the executable route requires constructing and justifying its monitor form. The intermediate assertion translation has a separate unresolved baseline diagnostic and is excluded from this expressiveness comparison.

III. Instance Semantics and Guarantees

The semantic object is an execution domain together with selected observations. We first define its meaning, then state the contracts that connect symbolic execution and proof checking to an instance verdict.

A. Instances and observations

A concrete configuration σ contains the control location, C environment, heap, and call stack. A logical environment η assigns values to typed logical variables. We write $z = (\sigma, \eta)$; η retains the root's entry values throughout execution. The interpretation Γ supplies the logical definitions, memory predicates, and target layout. Assertions are predicates on z, with separating conjunction interpreted by disjoint owned resources as in separation logic [6].

Definition 1 (Specification instance)**.** *An instance is $I = \langle c, R, E, \theta, \mathcal{B}, \Gamma, b\rangle$, where c is the C body, R and E are its entry and return assertions, θ is a typed partial binding, $\mathcal{B}$ selects observation boundaries, and b bounds execution and proof-search resources. Its input domain is*

$$D_\theta = \{(\sigma, \eta) \mid \eta \models \theta \wedge (\sigma, \eta) \models_\Gamma R\}. \tag{3}$$

A binding constrains selected values without choosing all unbound values. For example, $x = 13$ leaves the owned cell's address symbolic. Entry existence asks whether $D_\theta \neq \varnothing$. Subsequent execution checking quantifies over the entire domain. An existence witness therefore does not replace the domain by one concrete heap. Stronger bindings induce smaller domains, a property proved in `InstanceChecking.v`.

An observation resolves the selected assertion against the current C configuration and its immutable logical entry bindings. Let $\mathcal{B}_\Gamma(z)$ denote the finite list of resolved observations at z. For a finite execution trace $\tau = z_0 \cdots z_m$, define

$$\mathsf{Observe}_\mathcal{B}(\tau) = \bigoplus_{i=0}^{m} [(z_i, A) \mid A \in \mathcal{B}_\Gamma(z_i)],$$

where $\bigoplus$ concatenates lists, retaining order and multiplicity. A checked trace satisfies every recorded (z_i, A), has safe program transitions, and satisfies E at its terminal state. Calls introduce fresh logical bindings for the callee, keeping the caller's bindings available for return observations.

B. Symbolic execution and boundary rules

The symbolic configuration is $\langle \ell, P, K, U, \Omega, b\rangle$: a C control location, assertion describing possible states, call/continuation context, pending observation list, accumulated obligations, and remaining budget. Annotations are scheduled alongside C locations; consuming an observation advances U while retaining the C control location. A concrete step relation $\to_c$ supplies the program semantics. A symbolic step producing successors $P_1, \ldots, P_k$ must establish progress, safety, and coverage:

$$\begin{aligned} &\forall z \in [\![P]\!].\ \mathsf{Safe}(z) \wedge \exists z'.\ z \to_c z', \\ &\forall z, z'.\ z \in [\![P]\!] \wedge z \to_c z' \Rightarrow z' \in \bigcup_j [\![P_j]\!]. \end{aligned} \tag{4}$$

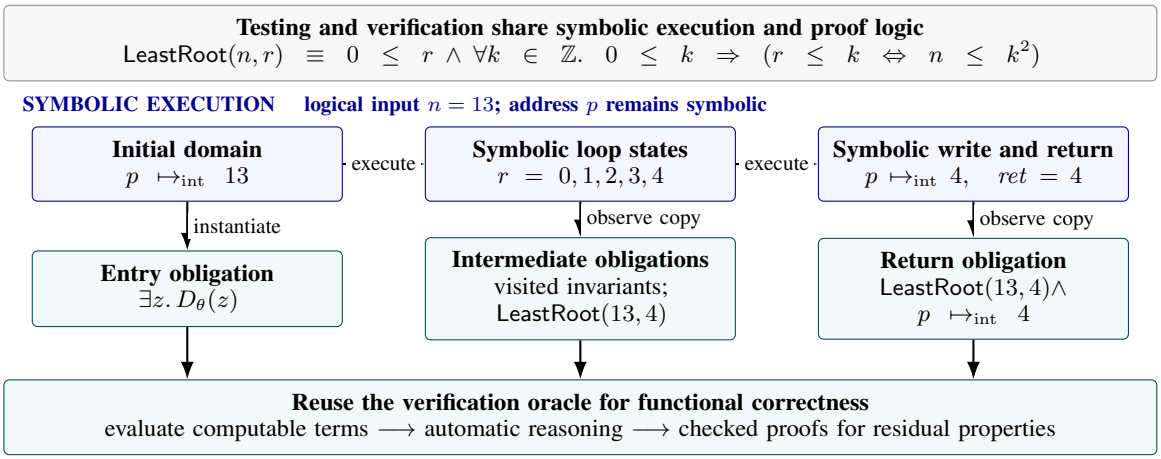
Here and below interpretations use Γ. Coverage permits an overapproximation for acceptance. The negative-result interface additionally requires that its admitted path states are realizable. An unresolved branch condition retains all successors whose infeasibility has not been established.

At an observation, the administrative rule is

$$\frac{Q = \mathsf{Resolve}_{\Gamma,K}(A) * F_{P,A}}{\langle P, A :: U, \Omega\rangle \xrightarrow{\text{observe}} \langle P, U, \Omega \cup \{P \vdash Q\}\rangle}. \tag{5}$$

The omitted context (ℓ, K, b) and represented program state P remain unchanged. The exact local-storage frame $F_{P,A}$ accounts for live scalar stack cells outside the observation's footprint (Section IV-B). Crucially, $P \vdash Q$ is recorded as an obligation, rather than required to enable the continuation.

The other administrative rules have explicit interfaces. Call entry copies evaluated arguments to fresh parameter slots, saves the caller context, and enters the actual body. Return retains the actual return value and shared writes, retires callee-local storage, restores the caller context, and records the selected callee postcondition on a checking copy. Loops repeatedly apply the condition and body transitions, recording observations at visited invariant locations. An exhausted execution budget creates an unfinished leaf.



Changing the intermediate assertion to LeastRoot$(13, 3)$ changes its proof obligation.
The program still writes and returns 4. Quantified properties retain their original logical form.

Fig. 1. Testing a functional specification in its verification logic. The upper path computes the program states; the lower path checks their logical properties. Copied-state observations preserve the computation when an annotation changes. The proof-capable oracle retains quantified properties after input specialization, using the same definitions and lemmas as verification.

C. Observation noninterference

The reference observation semantics records obligations independently of their truth. Its state already includes the program control location, so a boundary schedule can distinguish entries, returns, and individual loop visits. `ObservationTrace.v` defines ordinary finite executions and monitored executions with the same state trace and an additional observation list.

Theorem 1 (Erasure and lifting)**.** *For any resolved finite boundary schedule $\mathcal{B}$ and finite trace τ from z to z', there exists a monitored execution with state trace τ if and only if τ is a program execution from z to z'. Its observation list is exactly* $\mathsf{Observe}_{\mathcal{B}}(\tau)$.

Proof. Induct on the finite execution derivation. A zero-step trace records the observations of its sole state. A step preserves the concrete transition, prepends that state's observations, and applies the induction hypothesis to the remaining trace. Conversely, erasing the observation list from a monitored derivation preserves every program transition. No constructor requires a recorded assertion to hold. List induction establishes the observation inventory. □

Thus changing an assertion's truth cannot prune or create a program trace in the reference model. If all recorded obligations are discharged, each selected assertion holds at each corresponding visited state. Both directions and the observation-coverage property are mechanized.

D. Acceptance and negative evidence

A finite symbolic execution tree has program-step, normalization, observation, terminal, and unfinished nodes. Terminal nodes require safety, the return assertion, and the absence of further program transitions. Program-step nodes satisfy Equation (4); observation nodes follow Equation (5). Normalization preserves the represented joint observations (Section IV-C). An unfinished node has no acceptance certificate.

Theorem 2 (Nonvacuous instance acceptance)**.** *If the root represents D_θ, the domain is nonempty, and every obligation of a finite covering execution tree is valid, then every initial state in D_θ has safe, terminating executions satisfying E. Under the boundary coverage interface, every selected observation on those executions also holds.*

Proof. Induction on the tree establishes safe termination. A step covers every concrete successor and applies the induction hypothesis to each covering subtree. Normalization transfers membership through its semantic equivalence; observation preserves the continuation state. Terminal obligations establish the postcondition. Entry existence supplies nonvacuity independently. For observations, Theorem 1's inventory and checked boundary entailments establish satisfaction at each selected visit. □

The execution and nonvacuity result is mechanized in `InstanceChecking.v`; observation coverage is proved separately in `ObservationTrace.v`. The implementation obligations corresponding to these interfaces are collected in Section IV-D.

A confirmed violation has an existential meaning:

$$\exists z_0 \in D_\theta, z.\ z_0 \to_c^* z \wedge \mathsf{Bad}_{E,\mathcal{B}}(z). \tag{6}$$

Here badness is an unsafe state, a violated selected observation, or a terminal state violating E. The test executor preserves the entry constraints, branch conditions and actual state updates. For a path VC $P \vdash A$, SMT/ATP may discharge routine positive obligations or supply a candidate satisfying $P \wedge \neg A$. The agent may attempt either a proof of the original VC or a proof of its negation; both are checked against the frozen obligation. A checked positive proof establishes VALID. A

TABLE II
ENTRY RESULTS, VC STATES, AND WHOLE-TEST VERDICTS.

Layer	Result and meaning
Entry	SAT: the instance domain is nonempty. UNSAT: it is empty. UNKNOWN: satisfiability is unresolved.
VC	VALID: established. INVALID: an admitted path check establishes a violation. UNKNOWN: unresolved.
Test: PASS	Entry SAT, complete execution, and all selected obligations VALID.
Test: FAIL	Entry UNSAT, or a selected path VC INVALID; other obligations may remain unresolved.
Test: UNKNOWN	Neither complete acceptance nor a confirmed failed check.

checked negative proof with an admitted reachable path witness establishes INVALID. If neither direction is established, the VC remains UNKNOWN. Infeasible paths cannot cause a failed test merely because an implication proof was unfinished.

Table II separates logical results from execution bookkeeping. The report exposes `precondition_result`, per-obligation `vc_results`, and `test_result`. An UNSAT entry contributes a false result to the conjunction and stops execution. Infrastructure errors are reported separately. Legacy run-status fields remain available for compatibility. Unsupported logical encodings retain their original obligations for proof completion. Execution correspondence is part of the executor's trusted interface: an overapproximated proof-search state alone cannot license an INVALID path result.

The generated initial VC uses a negative-entailment encoding; its equivalence to existence is proved under an explicit classical logic premise in `PreconditionOracle.v`. Execution-only and Require-only checks retain different scopes even when both are reported as PASS by the command-line interface.

IV. CHECKING STATEFUL INSTANCES

The implementation connects QCP's execution and reasoning infrastructure through three interfaces: an instantiated entry domain, observations of program-derived states, and immutable obligations for proof completion. Figure 1 shows the separation between the execution and evidence paths.

A. Entry domains and actual execution

Typed bindings specialize the root specification. Logical entry values remain immutable as C variables and memory change. Recursive calls instantiate the original reusable contract with fresh logical variables and the current actual arguments. This distinction lets a root binding fix a recursion counter while subsequent calls decrease it normally.

The entry checker simplifies the instantiated Require and checks value ranges, binding consistency, and separation of owned cells. A faithful automatic encoding can establish nonemptiness. For an unresolved spatial predicate, the controller exports an existential Rocq obligation in the selected memory interpretation. Entry existence and execution coverage are checked separately: an existence witness supplies one model, while execution retains the entire symbolic domain.

The test reuses QCP to evaluate C expressions and update the symbolic heap. Logical bindings and ownership assertions supply the input without constructing a native heap or compiling an instrumented test binary. Known guards select a successor; unresolved guards retain every successor whose infeasibility has not been established. Calls evaluate actual arguments, allocate fresh parameter slots, and enter available bodies. Pointer arguments preserve the identity of shared storage. On return, the executor keeps the actual result and heap writes, retires local storage, and restores the caller context. Loops execute their conditions and bodies, including break, continue, and early return. An exhausted budget records an unfinished leaf. A declaration-only target supports an explicitly scoped Require-only check.

B. Observations on checking copies

At a selected boundary, the executor copies P, resolves the annotation, and emits $P \vdash A * F_{P,A}$. Execution continues with P regardless of whether the proof closes. Callee preconditions and postconditions are matched on such copies; the actual body supplies the caller's successor state. Invariant observations use each visited loop state.

a) Exact local frames.: QCP represents scalar C locals as owned cells. An auxiliary annotation can omit live local slots outside its footprint. The checker resolves goal-local witnesses using the current program-id/address map, then places precisely those unmentioned slots in $F_{P,A}$, preserving their types and values. User heap resources remain part of the checked footprint. If a local pointer holds $\&i$, resolving its value before choosing the frame exposes the i cell and prevents framing it twice.

Let m identify admitted local addresses and split h into h_m and $h_{\neg m}$. The scalar-cell model establishes, for extensional A,

$$(A * \mathit{exact}(h_m))(h) \iff A(h_{\neg m}).$$

With a goal witness w, the mask can depend on that witness; the theorem extends to $\exists w.\, (A_w * \mathit{exact}(h_{m_w}))(h)$. Checking one chosen witness proves an existential goal when successful; a failure for that witness leaves the existential obligation unresolved.

b) Boundary coverage.: The two compatibility policies, `expand` and `no-expand`, execute the same available bodies and loops. Both check entry existence, safety, and the target return. The second additionally observes selected internal annotations. An unlabelled annotation with several named alternatives generates their complete disjunction. Explicit routing retains the selected alternative. Unresolved matching and opaque local resources are recorded as coverage gaps.

C. Joint heap and value normalization

The continuation can refer to logical values outside the current heap formula. Eliminating r from $\exists r.\ r = 2 \wedge \mathit{cell}(p, 1)$ while leaving a return term that refers to r loses its value constraint. The normalization interface therefore preserves the

assertion P together with every live term $\bar{t}$, including results and saved call values. Define

$$\mathrm{Obs}(P, \bar{t})(s, \bar{v}) = \exists \eta.\ P(s, \eta) \wedge \bar{v} = \mathrm{eval}(\bar{t})_{s,\eta}.$$

Each admitted rewrite preserves this relation. Its composition consequently preserves the observations used by entry and boundary checks.

```
Normalize(Gamma, P, liveTerms, budget):
  while budget > 0 and an admitted reduction exists:
    resolve its definition and interpretation
    establish its guard; freshen typed local
    witnesses
    replace the occurrence by the complete
    equivalent body
    update dependent liveTerms consistently
    simplify exposed computations
    budget := budget - 1
  return (P, liveTerms)
```

Listing 2. Normalization of the state and its live values.

a) Admitting reductions.: Closed computations and known constructors expose reducible expressions. Definition unfolding retains every pure conjunct, spatial resource, and typed witness. For a guarded clause, activation requires $g(\bar{a}) \Rightarrow (H(\bar{a}) \equiv \exists \bar{w}.\, B(\bar{a}, \bar{w}))$ in the selected interpretation. The executor establishes g from the current state. Unresolved guards retain the predicate application. Budgeted rewriting terminates while preserving the meaning of residual terms.

For optional external rules, the controller checks the equation and its association with the program's qualified definitions, constructors, spatial notation, and layout. This prevents a same-named imported definition from changing the meaning of an accepted equation. Imported predicates otherwise use QCP's existing strategies. A checked pure consequence can be exposed by $H \equiv H \wedge G$, retaining the original spatial predicate and its witnesses. This path supports local proof progress without requiring a full layout translation.

b) Preserving witnesses and symbolic addresses.: Existing live variables remain protected during expansion. An unused witness of nominal type T requires inhabitance: $(\exists w : T.\, P) \equiv (\mathit{inhabited}(T) \wedge P)$ when w is absent from P. Integer and list types supply canonical witnesses; an arbitrary nominal type retains its existence obligation. Shared-base offset reductions and closed list slices simplify operations with symbolic pointers. Their ordinary C bounds and arithmetic safety obligations remain in the VC interface.

D. Proof completion and evidence

Automatic entailment and SMT reasoning attempt the generated obligations. Residual goals remain Rocq propositions. An optional proof agent or a developer constructs proofs; a fixed controller checks them against the frozen original module interface. The manifest binds source, logical dependencies, target interpretation, goal identities, and proof polarity. Whole-instance proof completion replays every original obligation, including entry existence and automatically discharged goals, audits transitive assumptions, and runs an independent kernel check. Execution errors and unfinished leaves retain their execution status after proof replay.

The assumption audit checks the logical environment as well as proof coverage. In both examined QCP runtimes, an inverse-field-address axiom together with evaluation and congruence axioms admits a contradiction by equating distinct syntax constructors. The retained adversarial proof compiles and is then rejected by the admission policy. This probe establishes why kernel checking and an explicit assumption policy are both part of the certificate interface.

a) Guarantee boundary.: The mechanized results cover the reference observation schedule, finite execution-tree interfaces, scalar frames, and joint normalization relation. The implementation must realize those interfaces through C expression semantics, target layouts, complete successor generation, and correct VC emission. Automatically accepted results additionally trust the executor and solver. A kernel certificate checks the exported statements in the admitted interpretation. These dependencies define the interpretation of each reported result.

E. Result aggregation

Figure 2 distinguishes proof polarity from an execution violation. A positive proof closes its original VC. For a path obligation $P \vdash A$, a negative proof or a model of $P \wedge \neg A$ establishes a violation only when the admitted path evidence supplies reachability from D_θ. The executable negative checker uses a private copy of the original path state, retaining pure constraints and owned scalar cells. Unsupported encodings and unresolved existential choices retain their obligations.

Entry SAT and valid VCs contribute true; entry UNSAT and admitted path violations contribute false; unresolved obligations and incomplete execution contribute unknown. The test conjunction returns PASS when all components are true, FAIL when any component is false, and UNKNOWN otherwise. An empty entry has the explicit reason "unsatisfiable entry" and stops execution. Historical report labels are preserved in the evaluation: INVALID denotes an empty entry there, and a checked VC refutation without reachability remains separate proof evidence.

V. Evaluation

We evaluate three questions: Can the original verification assertion serve as an instance oracle? Does copied-state checking retain the information required by later observations? What evidence and cost are associated with a complete result? The study combines matched contract configurations, independently checked proof groups, and development regressions. All numerical results below come from archived runs whose sources, goal interfaces, and hashes are checked by the paper build.

A. Design and comparison units

The paired archive contains 102 tool invocations covering 11 contract configurations from 10 C body families. Each configuration has a valid and mutated postcondition. CN

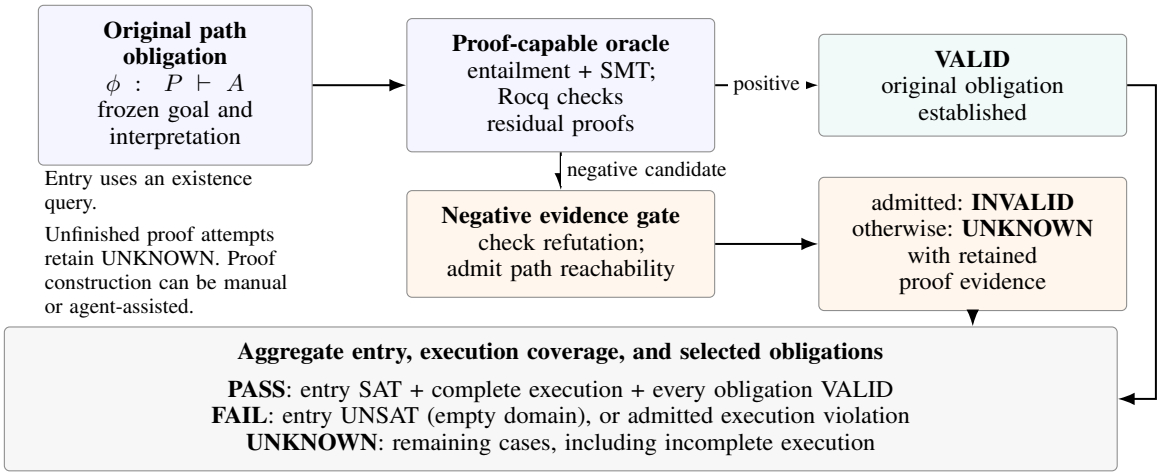


Fig. 2. Evidence required for a test verdict. Proof polarity is checked against the original obligation. A path-VC refutation requires admitted reachability evidence to yield a confirmed execution violation. Unfinished execution remains an unknown component. The entry gate reports an empty domain separately from a violated execution property.

TABLE III
INDEPENDENT CHECKING OF COMPLETE ORIGINAL VC GROUPS.

Case	VCs	Evidence	Check (s)
Cell increment	4	Proved	135.57
Square relation	3	Proved	129.53
Relational loop	44	Proved	137.47
Least-root return	26	Proved	132.06
List traversal	19	Proved	140.12
Least-root observation	27	Proved	124.45
Observation mutant	27	Refuted VC	125.86

Time includes certificate compilation, assumption audit, and kernel replay. These are separate archived runs; construction time is excluded. The mutant proves one VC's negation and the other 26 original obligations.

performs universal verification, Fulminate checks a concrete fixture, and TESPEC checks matched logical data with symbolic addresses. Repeated boundary policies are collapsed when their outcomes agree. We use CN to examine the effect of modular summaries and Fulminate to compare instance verdicts. Bennet and Darcy additionally test the translation of the least-root property. The original formula and its proved finite rewrite are reported separately.

A complete instance is the primary outcome. We also report phase results, residual obligations, proof polarity, and replay cost. Archived UNKNOWN results retain that status. Historical INVALID labels denote empty entry domains; ERROR labels denote execution or infrastructure diagnostics. The newer VC interface in Section IV-D does not change archived measurements.

B. Checking original mathematical assertions

The least-root return fixture fixes $n = 13$ and leaves the cell address symbolic. It executes four loop iterations and generates 26 obligations. Adding the quantified intermediate observation in Listing 1 yields a separate 27-obligation instance. Both complete proof groups pass independent checking, including entry existence and the original safety and functional goals. The observation instance receives an overall PASS after proof completion.

The pinned Fulminate, Bennet, and Darcy configurations reject the original quantified postcondition with an upper-bound inference diagnostic. The proved finite formulation in Equation (2) passes Fulminate on the same fixture. This experiment demonstrates the ability to retain the original assertion and locates the alternative's extra task: constructing an executable reformulation and justifying its equivalence. The generator attempts for the rewritten formula and the unresolved CN intermediate-annotation translation remain in the archive; the successful comparative result concerns the postcondition.

a) An assertion mutation with unchanged computation.: The negative instance replaces $\mathsf{LeastRoot}(n, r)$ with $\mathsf{LeastRoot}(n, r-1)$ before the write. Both variants execute four iterations, visit five invariant states, write 4, and return 4. The negative proof checks the negation of the complete assertion VC, including its generated alternatives, and proves the other 26 obligations. Its recorded result is a checked VC refutation. The archive has no admitted execution-counterexample certificate for that proof, so the evidence stops at that scope. This pair tests the assertion independently of the final output.

Table III also includes cell, square-relation, relational-loop, and list-traversal instances. The seven groups contain six positive groups and one refuted-VC group. They check original interfaces of 3–44 obligations; certificate compilation, assumption audit, and kernel replay take 124.45–140.12 seconds per group. These timings exclude proof construction. Proofs were obtained during development using supplied lemmas and exploratory proof work; agent success rates are a separate measurement.

C. Common-fragment results and execution preservation

On the paired common-fragment configurations, automatic TESPEC checking accepts 10 of 11 valid contracts and leaves

TABLE IV
PAIRED SUBJECTS: VALID / MUTATED CONTRACT OUTCOMES.

Subject	CN	Fulminate	TeSpec
alias_shared	V / U	P / F	P / U
array_cells	V / U	P / F	P / U
heap_loop	V / U	P / F	P / U
list_length	V / U	P / F	U / U
nested_call	V / U	P / F	P / U
pointer_cell	V / U	P / F	P / U
recursive_heap	V / U	P / F	P / U
recursive_heap_weak	U / U	P / F	P / U
single_cell	V / U	P / F	P / U
struct_fields	V / U	P / F	P / U
swap	V / U	P / F	P / U

V: verified; P: pass; F: runtime violation; U: unproved or unknown. CN checks full domains, Fulminate concrete fixtures, and TeSpec bound symbolic domains. The 11 contract configurations cover 10 body families; repeated modes are collapsed.

TABLE V
MECHANISM PROBES SEPARATE COMPLETE RESULTS FROM LOCAL PROOF PROGRESS.

Mechanism	Observed change
Local witness resolution	12 valid-mode runs: U $\rightarrow$ P; 12 incorrect-mode runs: U $\rightarrow$ U
Checked pure facts	Two configurations: 2 $\rightarrow$ 1 residual roots each; 0 new complete passes
Full proof completion	Positive least-root: U $\rightarrow$ P, all 27 obligations checked
VC refutation	Intermediate mutant: checked VC refutation; reachable violation unestablished

all 11 mutations UNKNOWN. Fulminate accepts all 11 valid fixtures and reports a runtime violation for every mutant. The list configuration requires residual proof work in TESPEC; a separate 19-obligation traversal certificate is reported in Table III. The paired automatic row retains its original outcome. Thus direct runtime checking provides more complete automatic negative feedback on this fragment. The proof-backed route supplies the mathematical-assertion capability measured above at the cost of residual proof work.

a) Weak recursive summaries.: The recursive-cell fixture starts at value 3 with counter 3 and returns the exact final value 6. Replacing the helper's exact postcondition with an existential value bounded below by the input leaves actual writes unchanged. Both TESPEC and Fulminate retain a passing result. CN's modular verifier and an unmodified QCP baseline retain the top-level exact-result obligation because the helper summary is too weak. This probe identifies the effect of executing the helper body while observing its contract. The exact and weak rows are two configurations of one body.

b) Local observations and normalization.: Resolving local pointer witnesses before choosing the exact local frame changes 12 valid-mode runs from UNKNOWN to PASS; the 12 incorrect-mode runs remain UNKNOWN. The 48 before/after observations cover 12 cases in two modes. For two configurations of one imported list instance, checked pure consequences reduce residual root obligations from two to one. Entry existence remains unresolved, yielding zero additional complete passes. Cold controller costs are 129.41 and 130.28 seconds. These probes distinguish successful local simplification from complete instance acceptance.

TABLE VI
DEVELOPMENT REGRESSION OUTCOMES (1,322 RUNS).

Verdict	Runs	Interpretation
PASS	541	Recorded checks discharged
UNKNOWN	757	Unresolved or incomplete
ERROR	14	Execution/coverage diagnostic
INVALID	10	Empty entry domain

All recorded expectations are met. Negative checks include deliberately unsupported cases; expectation agreement is distinct from confirmed violation detection.

D. Engineering coverage and formal evidence

The development suite contains 52 functional programs, each with four inputs, three variants, and two boundary policies, plus 37 diagnostic scenarios in both policies: $52 \cdot 4 \cdot 3 \cdot 2 + 37 \cdot 2 = 1{,}322$ checks. Functional expectations use independent Python oracles cross-checked against defined native C executions. All recorded expectations are met, with the outcomes in Table VI. The 779 expected negative checks include unsupported scenarios; their acceptance criterion permits UNKNOWN. Consequently, the suite measures avoidance of false acceptance and diagnostic conformance, while confirmed violations are counted separately. Each body mutation changes at least one observed outcome relative to its reference.

A broader historical scan covers 267 C files and 1,147 specifications with 5,160 synthetic-input runs: 1,086 PASS, 1,022 INVALID, 2,416 UNKNOWN, and 636 ERROR. Another 287 cases require typed bindings. A later default-strategy scan has a different denominator: 81 sources, 327 specifications, and 1,404 runs. It reports 448 PASS (212 execution and 236 specification-only), 726 UNKNOWN, 128 INVALID, and 102 ERROR. These inventories expose integration barriers such as missing bodies and opaque resources. Their synthetic inputs have no independent correctness labels. On eight original sort, queue, and reverse instances, the default workflow produces six UNKNOWN and two ERROR results. These outcomes locate the remaining execution and proof barriers in imported libraries.

The current formal replay checks 11 modules and audits 88 principal statements, including observation erasure, lifting, and coverage. The reference results support the semantic interfaces in Section III. Complete instance certificates separately check the generated obligations listed in Table III. Extended implementation probes and their unsuccessful attempts are retained in the supplementary development record.

VI. DISCUSSION

a) Where the capability pays off.: A developer with proof-oriented contracts and supporting lemmas can specialize logical

data and reuse the verifier to execute and check the instance symbolically. Symbolic addresses and owned resources supply the test heap; the workflow requires no native allocation harness. The least-root instance illustrates the tradeoff: proof completion retains the quantified formula; runtime checking uses a finite reformulation. On the common executable fragment, Fulminate provides complete negative feedback for the paired mutants, whereas TESPEC's automatic results remain unresolved. Proof construction and replay are therefore central costs of the additional oracle capability.

b) Contract debugging and contract quality.: The supplied Require and binding define D_θ. Comparing candidate specifications requires a behavior population fixed independently of those candidates: an over-strong Require could otherwise remove problematic inputs. Such a study should freeze initial heaps and positive/negative labels, record entry rejection, and check negative examples with the intended polarity. The current evaluation measures instance-checking capability and mechanisms.

c) Validity and transfer.: The programs and proof groups were selected during development. Common-fragment pairs control logical input data and C bodies; the least-root formula isolates one assertion-translation barrier. These results establish the reported capabilities on those subjects. Their frequency and proof effort across projects need held-out predicate families and programs. Imported predicates can still block reads or leave entry existence unresolved, as the original-library runs show. Complete C-to-VC refinement remains an implementation obligation beyond the mechanized reference interfaces.

Native differential runs use defined C executions. Architecture-sensitive comparisons require matched widths, layouts, ownership, and aliasing: the packaged executor models a 32-bit target while the native host is x86-64. The original allocator reproduction is retained in the supplementary record with its own budgets and target conditions. It supplies baseline artifact coverage independently of the paired results.

The archived timings are single runs, some concurrent with other proof work. They separate replay from construction cost and support the reported absolute measurements. Comparative speed and proof-agent success require repeated isolated runs, fixed libraries, and fixed budgets, as specified in the study protocol. Unsuccessful proofs, unsupported inputs, and unresolved intermediate baseline translations remain in the evidence inventory.

VII. RELATED WORK

a) Executable separation-logic specifications.: Separation logic describes disjoint ownership of mutable structures [6]. CN verifies systems C using separation-logic refinement types [7]; Fulminate translates ownership and functional assertions into runtime checks [1]. Bennet generates compatible heap inputs [2]. Code-Specify-Test-Debug-Prove integrates partial contracts, loop-invariant and lemma testing, and ghost arguments [3]. The CN workflow already unifies specification syntax across testing and verification and supports intermediate checks. Its testing path executes an instrumented binary on concrete heaps; TESPEC reuses symbolic execution to explore a partially instantiated logical heap domain. TESPEC extends the shared-specification workflow to a proof-capable test oracle for assertions that remain mathematical propositions after specialization: residual formulas over the symbolic heap are checked in their original proof logic. The least-root comparison makes this distinction concrete through an original formula and a proved executable rewrite. SLICK supplies runtime separation-logic checking [8], and CSF uses separation-logic preconditions for concolic heap testing [9]; these approaches further establish heap-aware execution as shared ground.

b) Testing declarative properties..: QuickCheck tests executable properties [10]; QuickChick connects property-based testing to foundational proofs and derives generators and semi-decision procedures for supported inductive relations [11], [12]. COINS specializes Rocq specifications on trusted examples and checks concrete proof obligations [4]. TESPEC connects this style of oracle to program-derived symbolic heaps and selected internal boundaries, with explicit execution coverage and observation preservation. VERINA evaluates specifications through positive and negative tests [13]; SpecRL uses rejected negative behaviors, including materialized side effects, as a completeness signal [14]. FormalBench and MutDafny assess specification strength using mutation [15], [16]. Candidate-quality evaluation additionally requires the independently fixed behavior population discussed in Section VI.

c) Symbolic verification and proof infrastructure.: QCP already combines symbolic execution, separation-logic reasoning, and interactive Rocq proofs [5]; CN also has a Rocq lemma translation [17]. TESPEC reuses this proof infrastructure. Its instance discipline executes bodies and loops and checks candidate summaries on private copies, preserving the actual state for later observations. Joint heap/value normalization and exact local frames maintain that discipline through state representation changes. The proof controller then checks the complete original instance interface and its admitted assumptions. This combination connects partial input domains, stateful observations, and scoped evidence for each complete result.

VIII. CONCLUSION

TESPEC formulates stateful specification testing as instance-specialized symbolic verification, reusing the verifier's execution and proof infrastructure. A functional assertion can serve as a test criterion even when its instance requires proof. Partial bindings determine the input domain, program transitions determine the states, and copied-state observations determine the proof obligations. Joint normalization and exact local frames preserve the values and resources needed by later checks. The least-root cases demonstrate direct checking of a quantified return property and an intermediate assertion, while the paired common-fragment results expose the cost in residual proof work. The resulting method connects proof-oriented instance evaluation to the internal specification boundaries of C.

Data Availability

The accompanying artifact contains the controller, executor build identities, logical runtime, regression reports, certificate sources, and Rocq semantic modules. The paper's evidence inventory records the source of each reported result and distinguishes archived measurements from the current theory replay. A build script regenerates this manuscript and its vector overview figure.

References

[1] R. Banerjee, K. Memarian, D. Makwana, C. Pulte, N. Krishnaswami, and P. Sewell, "Fulminate: Testing CN separation-logic specifications in C," *Proceedings of the ACM on Programming Languages*, vol. 9, no. POPL, pp. 1260–1292, 2025.

[2] Z. K. Aamer and B. C. Pierce, "Bennet: Randomized specification testing for heap-manipulating programs," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 3924–3953, 2025.

[3] Z. K. Aamer, R. Banerjee, H. Katsura, D. Kaloper-Meršinjak, D. J. Economou, K. Memarian, D. Makwana, N. Krishnaswami, B. C. Pierce, C. Pulte, and P. Sewell, "Code-specify-test-debug-prove: Flexibly integrating separation logic specification into conventional workflows," *Proceedings of the ACM on Programming Languages*, vol. 10, no. PLDI, 2026.

[4] F. Yang, X. Li, S. Wang, J. An, Z. Sun, S. Feng, W. Wang, W. Wang, N. Zhan, and F. Xu, "How powerful are LLMs in generating formal program specifications?" in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 145 221–145 249. [Online]. Available: https://proceedings.mlr.press/v306/yang26am.html

[5] X. Wu, Y. Feng, X. Lu, T. Lin, K. Liu, Z. Wang, S. Wu, L. Xie, C. Yang, H. Zhong, Z. Zhang, J. Li, N. Zhan, Z. Hu, and Q. Cao, "QCP: A practical separation logic-based C program verification tool," *arXiv preprint arXiv:2505.12878*, 2026. [Online]. Available: https://arxiv.org/abs/2505.12878

[6] J. C. Reynolds, "Separation logic: A logic for shared mutable data structures," *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science*, pp. 55–74, 2002.

[7] C. Pulte, D. C. Makwana, T. Sewell, K. Memarian, P. Sewell, and N. Krishnaswami, "CN: Verifying systems C code with separation-logic refinement types," *Proceedings of the ACM on Programming Languages*, vol. 7, no. POPL, 2023.

[8] H. H. Nguyen, V. Kuncak, and W.-N. Chin, "Runtime checking for separation logic," in *Verification, Model Checking, and Abstract Interpretation*, 2008, pp. 203–217.

[9] L. H. Pham, Q. L. Le, Q.-S. Phan, and J. Sun, "Concolic testing heap-manipulating programs," in *Formal Methods – The Next 30 Years*, 2019, pp. 442–461.

[10] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 2000, pp. 268–279.

[11] Z. Paraskevopoulou, C. Hritcu, M. Dénès, L. Lampropoulos, and B. C. Pierce, "Foundational property-based testing," in *Interactive Theorem Proving*, 2015, pp. 325–343.

[12] Z. Paraskevopoulou, A. Eline, and L. Lampropoulos, "Computing correctly with inductive relations," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022. [Online]. Available: https://lemonidas.github.io/pdf/ComputingCorrectly.pdf

[13] Z. Ye, Z. Yan, J. He, T. Kasriel, K. Yang, and D. Song, "VERINA: Benchmarking verifiable code generation," in *International Conference on Learning Representations*, 2026. [Online]. Available: https://arxiv.org/abs/2505.23135

[14] Z. Huang, Z. Zhang, Z. Sun, H. Sun, and Y. Xiong, "Reinforcement learning with negative tests as completeness signal for formal specification synthesis," *arXiv preprint arXiv:2604.05820*, 2026. [Online]. Available: https://arxiv.org/abs/2604.05820

[15] T. Le-Cong, B. Le, and T. Murray, "Can LLMs reason about program semantics? a comprehensive evaluation of LLMs on formal specification inference," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2025, pp. 21 991–22 014. [Online]. Available: https://aclanthology.org/2025.acl-long.1068/

[16] I. Amaral, A. Mendes, and J. Campos, "MutDafny: A mutation-based approach to assess Dafny specifications," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. [Online]. Available: https://arxiv.org/abs/2511.15403

[17] A. Yin, "Translating CN separation logic lemmas to the Rocq proof assistant," MPhil thesis, University of Cambridge, 2025. [Online]. Available: https://www.cl.cam.ac.uk/~nk480/amy-yin-cn-lemmata-mphil-thesis.pdf